RLHF

前言

1.强化学习基础

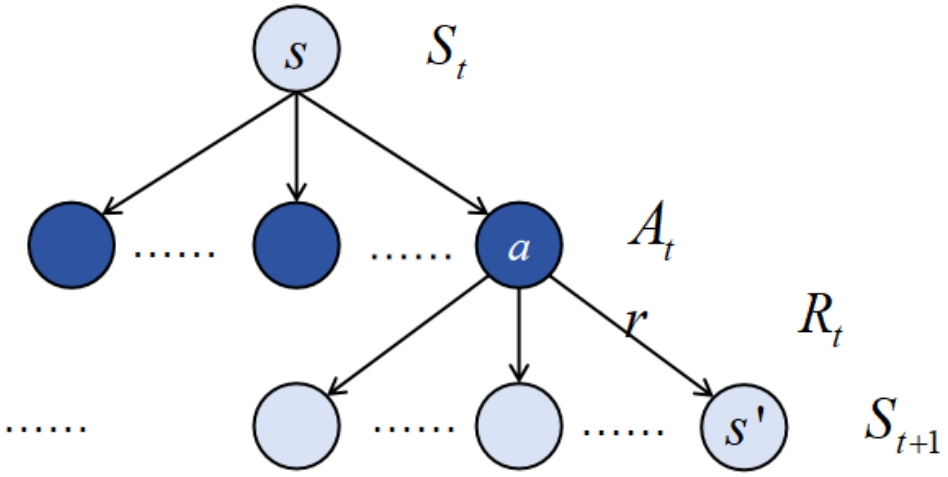
1.1 有限马尔可夫决策过程

1.2价值函数与优势函数

状态价值函数，在Sutton的著作中，将状态价值函数的定义如下：

$$v_\pi(s) \doteq \mathbb{E}_\pi\Big[G_t|S_t=s\Big] = \mathbb{E}_\pi\Big[\sum_{k=0}^{\infty}\gamma^k R_{t+k+1}\Big], \forall s \in \mathcal{S}$$

状态价值函数反映了从状态 $S_t = s$ 开始，按照策略 π执行动作后，智能体预计将获得的总奖励的期望值。



而价值函数满足某种递归关系，对于任何策略π和状态s，s的价值与其后继状态的价值关系如下：

$$\begin{aligned} v_\pi(s) &\doteq \mathbb{E}_\pi\Big[G_t|S_t=s\Big] \\ &= \mathbb{E}_\pi\Big[R_t+\gamma G_{t+1}|S_t=s\Big] \\ &= \sum_a \pi(a|s)\sum_{r,s'} p(r,s'|s,a)(r+\gamma v_\pi(s')) \end{aligned}$$

动作价值函数 则是根据策略π，从状态s开始，执行动作a之后所有可能的决策序列的期望回报，我们用符号$q_\pi(s,a)$表示：

$$\begin{aligned} q_\pi(s,a) &\doteq \mathbb{E}_\pi\Big[G_t|S_t=s, A_t=a\Big] \\ &= \sum_{r,s'} p(r,s'|s,a)(r+\gamma v_\pi(s')) \end{aligned}$$

值得注意的是回报R_t是一个简写，完整的表示应该是$R(S_t, A_t, S_{t+1})$。我们将智能体在策略pi控制下的预期折扣收益用符号$J(\pi)$表示：

$$J(\pi) = \mathbb{E}_\pi\Big[G_t\Big] = \mathbb{E}_\pi\Big[\sum_{t=0}^{\infty}\gamma^t R(S_t, A_t, S_{t+1})\Big]$$

公式变体，在某些强化学习论文中，作者会为了简化公式的表达或者让某个特定的推导更清晰，而对标准符号做一些变动。如省略某些下角标或者是加上下角标变量，亦或者是对公式进行拓展，使得读者在阅读不同论文或材料时感到困惑，在本文接下来的内容中会涉及到不同的论文公式推导，为方便读者理解，笔者先列举一些公式的变体：

$$\begin{aligned}
J(\pi) &= \mathbb{E}_\pi\Big[\sum_{t=0}^{\infty}\gamma^t R(S_t, A_t, S_{t+1})\Big] \\
&= \mathbb{E}_{s_0, a_0, \ldots}\Big[\sum_{t=0}^{\infty}\gamma^t R(S_t, A_t, S_{t+1})\Big] \\
&= \mathbb{E}_{\tau\sim(\pi,E)}\Big[G(\tau)\Big] \\
&= \sum_{\tau\sim(\pi,E)} P(\tau|\pi)G(\tau)
\end{aligned}$$

公式的改写本质上是对同一个问题的不同表示方式，如上式，第一二行通常用于标准的动态规划方法或分析中，直接表示在时间步t处从状态S_t开始采取动作A_t所获得的累积奖励的期望。第三四行则是从策略 π和环境E生成的轨迹序τ的角度出发。同样，状态价值函数与动作价值函数也可以进行适当改写：

$$\begin{aligned}
v_\pi(s) &\doteq \mathbb{E}_\pi\Big[G_t|S_t = s\Big] \\
&= \mathbb{E}_\pi\Big[R_t + \gamma G_{t+1}|S_t = s\Big] \\
&= \mathbb{E}_{S_t=s, A_t, \ldots}\Big[\sum_{t'=t}^{\infty}\gamma^{t'-t}R(S_{t'} = s, A_{t'}, S_{t'+1})\Big], \\
q_\pi(s,a) &\doteq \mathbb{E}_\pi\Big[G_t|S_t = s, A_t = a\Big] \\
&= \mathbb{E}_{S_t=s, A_t=a, \ldots}\Big[\sum_{t'=t}^{\infty}\gamma^{t'-t}R(S_{t'} = s, A_{t'} = a, S_{t'+1})\Big]
\end{aligned}$$

优势函数，

$$A_\pi(s,a) = \underbrace{q_\pi(s,a)}_{\text{On-Policy}} - \underbrace{v_\pi(s)}_{\text{On-Policy}}$$

2.RewardModeling

3.策略梯度算法

我们现在来看一下什么是策略梯度算法，以及其背后的动机与直觉。我们希望我们的策略能够使得回报的期望最大，动作的轨迹是在策略π_θ的控制下产生的，因此我们的目标函数可以表达如下：

$$
\begin{aligned}
J(\pi_\theta) &= \mathbb{E}_{\tau\sim(\pi_\theta,E)}\Big[G(\tau)\Big] \\
&= \sum_{\tau\sim(\pi_\theta,E)} P(\tau|\theta)G(\tau)
\end{aligned} \tag{2.1}
$$

其中E是环境，这个目标函数衡量了在指定环境下从我们的策略中采样的轨迹的理论收益。如果我们想找到最大化这个目标函数的参数θ，我们可以通过梯度上升的方式不断迭代更新θ。更新过程表示如下：

$$
\theta_{t+1} = \theta_t + \alpha\nabla_\theta J(\pi_\theta)|_{\theta_t} \tag{2.2}
$$

其中$\nabla_\theta J(\pi_\theta)|_{\theta_t} = \nabla_{\theta_t} J(\pi_{\theta_t})$，也就是所谓的**策略梯度**。现在的问题在于，我们如何计算$\nabla_\theta J(\pi_\theta)|_{\theta_t}$？利用策略梯度则需要一个明确的可计算的表达式。接下来我们进一步探索如何找到这个明确的表达式并加以运用。

1.轨迹的概率，给定策略π_θ下轨迹$\tau = (s_0, s_1, \ldots, s_{T+1})$出现的概率如下：

$$
P(\tau|\theta) = \rho(s_0)\prod_{t=0}^{T} P(s_{t+1}|s_t, a_t)\pi_\theta(a_t|s_t) \tag{2.3}
$$

2.Log-Derivative Trick，利用log求导的技巧$\partial_x \log(f(x)) = \frac{\partial_x f(x)}{f(x)}$，对轨迹概率关于$\theta$求导，我们有：

$$
\nabla_\theta P(\tau|\theta) = \nabla_\theta \log P(\tau|\theta)\cdot P(\tau|\theta) \tag{2.4}
$$

3.轨迹的对数概率，$P(\tau|\theta)$的对数概率如下：

$$
\log P(\tau|\theta) = \underbrace{\log(\rho(s_0))}_{\text{与}\theta\text{无关}} + \sum_{t=0}^{T}\underbrace{\log P(s_{t+1}|s_t, a_t)}_{\text{与}\theta\text{无关}} + \log(\pi_\theta(a_t|s_t)) \tag{2.5}
$$

4.轨迹对数概率的梯度，由于部分项与θ无关，所以$\nabla_\theta \log P(\pi|\theta)$表达如下：

$$
\begin{aligned}
\nabla_\theta \log P(\tau|\theta) &= \nabla_\theta \sum_{t=0}^{T}\log(\pi_\theta(a_t|s_t)) \\
&= \sum_{t=0}^{T}\nabla_\theta \log(\pi_\theta(a_t|s_t))
\end{aligned} \tag{2.6}
$$

将上述式子结合，求出$\nabla_\theta J(\pi_\theta)$则有：

$$
\begin{aligned}
\nabla_\theta J(\pi_\theta) &= \nabla_\theta \sum_{\tau\sim(\pi_\theta,E)} P(\tau|\theta)G(\tau) \\
&= \sum_{\tau\sim(\pi_\theta,E)} \nabla_\theta P(\tau|\theta)G(\tau) \\
&= \sum_{\tau\sim(\pi_\theta,E)} \nabla_\theta \log P(\tau|\theta)P(\tau|\theta)R(\tau) \\
&= \sum_{\tau\sim(\pi_\theta,E)} \sum_{t=0}^{T}\nabla_\theta \log(\pi_\theta(a_t|s_t))P(\tau|\theta)G(\tau) \\
&= \mathbb{E}_{\tau\sim(\pi_\theta,E)}[\sum_{t=0}^{T}\nabla_\theta \log(\pi_\theta(a_t|s_t))G(\tau)]
\end{aligned} \tag{2.7}
$$

也就是说，我们可以根据采样到的轨迹来计算策略梯度，即如果收集到了一系列由π_θ产生的轨迹$D = \{\tau_i\}_{i=1}^{N}$，我们可以利用如下公式估计策略梯度：

$$\hat{\nabla}_\theta J(\pi_\theta) = \frac{1}{|D|}\sum_{i=1}^{N}\sum_{t=0}^{T}\nabla_\theta \log(\pi_\theta(a_t|s_t))G(\tau) \tag{2.8}$$

如上便是策略梯度最简单的形式。如果我们能够在环境中运行策略来收集轨迹数据集，我们可以计算策略梯度并采取更新步骤。

Don't Let the Past Distract You.策略梯度的公式告诉我们对于一个轨迹$\tau = (s_0, s_1, \ldots, s_T)$，每一个时刻的动作执行后对应的策略梯度都会乘以一个因子，即回报$R(\tau)$，但这并没有什么意义，因为是否强化智能体当前的决策动作只因和当前动作执行后产生的影响有关，如果当前动作执行后的收益低则不应当强化当前动作，反之亦然，而不能受先前因素的影响。因此，对于回报$R(\tau)$我们可以做出适当改进，不再考虑当前时刻t'之前的收益，则策略梯度可以重写为：

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim(\pi_\theta,E)}[\sum_{t=0}^{T}(\nabla_\theta \log(\pi_\theta(a_t|s_t)\sum_{t=t'}^{T}R(s_{t'}, a_{t'}, s_{t'+1}))] \tag{3.10}$$

在这个形式下，动作只基于在采取行动后获得的奖励而得到强化。我们将这种形式称为 **“Reward-to-go”**，因为回报为在轨迹的一个点之后奖励的总和。

策略梯度算法有一些变种形式，这些变种都与我们先前所学习的内容有所关联，接下来我们学习一些变种算法从而对策略梯度算法有更深入的理解。

Baseline in Policy Gradients. 使用策略梯度面临着一个问题——准确地估算出梯度需要大量的样本轨迹，使用 EGLP 引理，我们可以证明**Reward -to-go**——尽管没有改变政策梯度的期望值——减少了我们估计的方差，因此减少了估计策略梯度所需的轨迹总数。而EGLP的直接结果是对于任何直接依赖于状态的函数b，我们有：

$$\mathbb{E}_{a_t\sim\pi_\theta}[\nabla_\theta \log(\pi_\theta(a_t|s_t)b(s_t)] = 0 \tag{3.11}$$

这使得我们可以在不改变期望值的情况下在策略梯度表达式上加上或减去任意项：

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim(\pi_\theta,E)}[\sum_{t=0}^{T}\left(\nabla_\theta \log(\pi_\theta(a_t|s_t)(\sum_{t=t'}^{T}R(s_{t'}, a_{t'}, s_{t'+1}) - \underbrace{b(s_t)}_{\text{Baseline}})\right)] \tag{3.12}$$

在这个表达式中，任何函数b都称作baseline。最常见的基线选择是状态价值函数$V^\pi(s_t)$。回想一下，这是一个智能体从状态开始，然后在其剩余生命周期内按照策略行事时获得的平均回报。从**经验上**讲，这种选择具有减少策略梯度样本估计方差的效果，使得策略梯度学习更快更稳定。从直观上将，假设在某个状态s_t下，智能体采取了一个动作 a_t，并得到了一个回报。如果这个回报远高于该状态的预期回报（即 $V^\pi(s_t)$），我们认为这个动作“更好”；如果低于预期回报，我们认为该动作“不好”，通过减去$V^\pi(s_t)$，我们将焦点集中在“超出预期的部分”（即优势）上。这就像在与基准相比时，我们只关注某个动作相对于平均策略的改进或削弱。

> **Note**
>
> 值得注意的是，$V^\pi(s_t)$并不能准确的计算，所以需要近似计算，通常我们用一个神经网络$V_\phi^\pi(s_t)$估计价值函数，它与策略网络同时更新，而V_ϕ^θ的优化目标通常是最小均方误差（包括$VPG, TRPO, PPO$等），即：
>
> $$\phi_k = \arg\min_\phi \mathbb{E}_{s_t,\hat{R}_t\sim\pi_k}\left[\left(V_\phi(s_t) - \hat{R}_t\right)^2\right] \tag{3.13}$$

我们可以以一种更一般地形式写出策略梯度：

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim(\pi_\theta,E)}\left[\sum_{t=0}^{T}\left(\nabla_\theta \log(\pi_\theta(a_t|s_t)\Psi(t)\right)\right] \tag{3.14}$$

$\Psi(t)=R(\tau)$时$\nabla_\theta J(\pi_\theta)$是基础，$\Psi(t)=\sum_{t=t'}^{T}R(s_{t'},a_{t'},s_{t'+1})$时是**Reward-to-go**，$\Psi(t)=\sum_{t=t'}^{T}R(s_{t'},a_{t'},s_{t'+1})-b(s_t)$是**Reward-to-go with baseline**。此外，$\Psi(t)$的选择还可以是动作价值函数$Q^\pi(s_t,a_t)$，优势函数$A^\pi(s_t,a_t)=Q^\pi(s_t,a_t)-V^\pi(s_t)$，利用优势函数的策略梯度的公式化极为常见，并且不同算法使用的优势函数有许多不同的估计方法。

3.1TRPO

为保证上下文符号一致，笔者在本章节推导上的符号并未遵循原论文， 进行了一定改动。

一个策略$\tilde{\pi}$关于另一个策略π的预期收益在累计时间步上的优势为：

$$J(\tilde{\pi})-J(\pi)=\mathbb{E}_{\tau\sim\tilde{\pi}}\left[\sum_{t=0}^{\infty}\gamma^t A_\pi(s_t,a_t)\right] \tag{3.15}$$

证明如下：

$$\begin{aligned}
&\mathbb{E}_{\tau\sim\tilde{\pi}}\left[\sum_{t=0}^{\infty}\gamma^t A_\pi(s_t,a_t)\right]\\
&\quad=\mathbb{E}_{\tau\sim\tilde{\pi}}\left[\sum_{t=0}^{\infty}\gamma^t(q_\pi(s_t,a_t)-v_\pi(s_t))\right]\\
&\quad=\mathbb{E}_{\tau\sim\tilde{\pi}}\left[\sum_{t=0}^{\infty}\gamma^t(R(s_t,a_t,s_{t+1})+\gamma v_\pi(s_{t+1})-v_\pi(s_t))\right]\\
&\quad=\mathbb{E}_{\tau\sim\tilde{\pi}}\left[\sum_{t=0}^{\infty}\gamma^t R(s_t,a_t,s_{t+1})+\gamma^{t+1}v_\pi(s_{t+1})-\gamma^t v_\pi(s_t)\right]\\
&\quad=\mathbb{E}_{\tau\sim\tilde{\pi}}\left[\sum_{t=0}^{\infty}\gamma^t R(s_t,a_t,s_{t+1})\right]+\mathbb{E}_{\tau\sim\tilde{\pi}}\left[\sum_{t=0}^{\infty}\gamma^{t+1}v_\pi(s_{t+1})-\gamma^t v_\pi(s_t)\right]\\
&\quad=J(\tilde{\pi})+\mathbb{E}_{\tau\sim\tilde{\pi}}\left[\sum_{t=1}^{\infty}\gamma^t v_\pi(s_t)-\sum_{t=0}^{\infty}\gamma^t v_\pi(s_t)\right]\\
&\quad=J(\tilde{\pi})-\mathbb{E}_{\tau\sim\tilde{\pi}}\left[v_\pi(s_0)\right]\\
&\quad=J(\tilde{\pi})-J(\pi)
\end{aligned} \tag{3.16}$$

如果能保证$J(\tilde{\pi})-J(\pi)$大于0，则能说明更新后的策略一直在进步，而优势函数这一项又可以改写成：

$$\begin{aligned}
&\mathbb{E}_{\tau\sim\tilde{\pi}}\left[\sum_{t=0}^{\infty}\gamma^t A_\pi(s_t,a_t)\right]\\
&\quad=\sum_{t=0}^{\infty}\sum_{s}p(s_t=s|\tilde{\pi})\sum_{a}\tilde{\pi}(a_t=a|s)\gamma^t A_\pi(s,a)\\
&\quad=\sum_{t=0}^{\infty}\sum_{s}\gamma^t p(s_t=s|\tilde{\pi})\sum_{a}\tilde{\pi}(a_t=a|s)A_\pi(s,a)\\
&\quad=\sum_{s}\sum_{t=0}^{\infty}\gamma^t p(s_t=s|\tilde{\pi})\sum_{a}\tilde{\pi}(a_t=a|s)A_\pi(s,a)\\
&\quad=\sum_{s}\rho_{\tilde{\pi}}(s)\sum_{a}\tilde{\pi}(a_t=a|s)A_\pi(s,a)
\end{aligned} \tag{3.17}$$

其中，$\rho_{\tilde{\pi}}(s)=p(s_0=s|\tilde{\pi})+\gamma p(s_1=s|\tilde{\pi})+,\ldots,$ $\tilde{\pi}$是之前的策略π更新后的新策略，上述式子中涉及到$p(s_t=s|\tilde{\pi})$与$\tilde{\pi}(a_t=a|s)$，即我们要按照新的策略与环境交互才能得到轨迹，先确定新的策略$\tilde{\pi}$并得到一定量的样本才能求解，并计算是否满足$\mathbb{E}_{\tau\sim\tilde{\pi}}\left[\sum_{t=0}^{\infty}\gamma^t A_\pi(s_t,a_t)\right]>0$。$TRPO$利用函数$\mathcal{L}_\pi(\tilde{\pi})$代替原始目标函数：

$$\mathcal{L}_\pi(\tilde{\pi})=J(\pi)+\sum_s\rho_\pi(s)\sum_a\tilde{\pi}(a\mid s)A_\pi(s,a). \tag{3.18}$$

只要策略更新的幅度不大，就可以用$\mathcal{L}_\pi(\tilde{\pi})$近似原本的$J(\tilde{\pi})$，所以那我们**怎么来保证其更新幅度不要太大呢**？为了解决这个求解信任区域的问题，文中引入了Kakade&Langford（2002）的结论——Conservative policy iteration：

$$\begin{aligned}\pi_{\text{new}}(a\mid s)&=(1-\alpha)\pi_{\text{old}}(a\mid s)+\alpha\pi'(a\mid s)\\ \eta(\pi_{\text{new}})&\geq L_{\pi_{\text{old}}}(\pi_{\text{new}})-\frac{2\epsilon\gamma}{(1-\gamma)^2}\alpha^2\\ &\text{where }\epsilon=\max_s\left|\mathbb{E}_{a\sim\pi'(a|s)}\left[A_\pi(s,a)\right]\right|\end{aligned} \tag{3.19}$$

有了这个下界表达式，我们可以利用**minorization-maximization**算法通过$\mathcal{L}_{\pi_{old}}(\pi_{new})$迭代$J(\pi_{new})$。该算法具体细节不在本文涉及范围内，值得注意的是，该原始结论只适合混合策略，但实际应用中的混合策略很少使用，因此作者将该结论拓展到了一般随机策略[x]。最终的优化目标变成：

$$\underset{\pi_\theta}{\text{maximize}}\left[\mathcal{L}_{\pi_{\theta_{\text{old}}}}(\pi_\theta)-CD_{\text{KL}}^{\max}(\pi_{\theta_{\text{old}}},\pi_\theta)\right]. \tag{3.20}$$

其中：

$$\mathcal{L}_{\pi_{old}}(\pi_\theta)=J(\pi)+\sum_s\rho_{\pi_{\theta_{old}}}(s)\sum_a\pi_\theta(a\mid s)A_{\pi_{\theta_{old}}}(s,a).$$

由于$\pi_\theta(a|s)$与新策略有关，无法对其直接采样，因此我们通过重要性采样的方式进行采样，我们将式子右边项进行变体：

$$\begin{aligned}&\sum_s\rho_{\pi_{\theta_{old}}}(s)\sum_a\pi_\theta(a\mid s)A_{\pi_{\theta_{old}}}(s,a)\\ &=\sum_s\sum_t\gamma^t p(s_t=s|\pi_{\theta_{old}})\sum_a\pi_\theta(a\mid s)A_{\pi_{\theta_{old}}}(s,a)\\ &=\sum_t\gamma^t\sum_s p(s_t=s|\pi_{\theta_{old}})\sum_a\pi_\theta(a\mid s)A_{\pi_{\theta_{old}}}(s,a)\\ &=\sum_t\gamma^t\mathbb{E}_{s\sim\rho_{old}}\left[\sum_a\pi_\theta(a\mid s)A_{\pi_{\theta_{old}}}(s,a)\right]\\ &=\frac{1}{1-\gamma}\mathbb{E}_{s\sim\rho_{old}}\left[\sum_a\pi_\theta(a\mid s)A_{\pi_{\theta_{old}}}(s,a)\right]\end{aligned} \tag{3.21}$$

$\sum_a\pi_\theta(a\mid s)A_{\pi_{\theta_{old}}}(s,a)$可以通过重要性采样的方式重新表述成：

$$\begin{aligned}\sum_a\pi_\theta(a\mid s)A_{\pi_{\theta_{old}}}(s,a)&=\mathbb{E}_{a\sim q}\left[\frac{\pi_\theta(a\mid s)}{q_(a\mid s)}A_{\pi_{\theta_{old}}}(s,a)\right]\\ &\mapsto\mathbb{E}_{a\sim\pi_{\theta_{old}}}\left[\frac{\pi_\theta(a\mid s)}{\pi_{\theta_{old}}(a\mid s)}A_{\pi_{\theta_{old}}}(s,a)\right]\end{aligned}$$

故最终的优化目标为：

$$\begin{aligned}&\arg\max_\theta\mathbb{E}_{s\sim\rho_{old},a\sim\pi_{old}}\left[\frac{\pi_\theta(a\mid s)}{\pi_{\theta_{old}}(a\mid s)}A_{\pi_{\theta_{old}}}(s,a)\right]\\ &\text{subject to }\mathbb{E}_{s\sim\rho_{old}}\left[D_{KL}(\pi_{\theta_{old}}(\cdot|s)||\pi_\theta(\cdot|s))\right]\leq\delta\end{aligned} \tag{3.22}$$

3.2 PPO

我们现在将深入理解为语言模型对齐奠定基础的算法 —— PPO(近端策略优化算法)，PPO是一种基于策略梯度的强化学习算法，PPO的核心思想是通过在每次更新时保持策略的“平稳性”或“稳定性”，避免过度优化，从而减少策略更新过程中的波动性，PPO算法的优化目标如下：

$$J(\theta)=\frac{1}{G}\sum_{i=1}^{G}\min\left(\frac{\pi_\theta\left(a_t \mid s\right)}{\pi_{\theta_{old}}\left(a_t \mid s_t\right)}A_t,\operatorname{clip}\left(\frac{\pi_\theta\left(a_t \mid s_t\right)}{\pi_{\theta_{old}}\left(a_t \mid s_t\right)},1-\varepsilon,1+\varepsilon\right)A_t\right)$$

注意到$\min$函数会使优化目标有不同取值的选择，我们来解释不同情况下的取值情况，首先定义策略比率如下：

$$R(\theta)=\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

当优势是正数且策略比率超过$1+\varepsilon$时，意味着新的策略更可能采取a_t行动，$\operatorname{clip}(\cdot)$通过裁剪比率防止策略更新过大，并限制$R(\theta)$的变化范围，此时优化目标：

$$J(\theta)=\min(R(\theta),1+\varepsilon)A_t=(1+\epsilon)A$$

意味着优化目标会增加策略比率，使得动作更可能发生，当优势是正数而策略比率小于$1-\varepsilon$时，优化目标变成：

$$J(\theta)=\min(R(\theta),1-\varepsilon)A_t=R(\theta)A_t$$

意味着优化目标会减小策略比率，使得动作没那么可能发生。同理，如果策略比率本身介于$(1-\varepsilon,1+\varepsilon)$之间，则有：

$$J(\theta)=\min(R(\theta),R(\theta))=R(\theta)$$

没有任何影响，当优势是负数时且策略比率$R(\theta)<1-\varepsilon$时，有：

$$J(\theta)=\min(R(\theta)A_t,(1-\varepsilon)A_t)=(1-\varepsilon)A_t$$

其他情况同理，在信任区域内的优化目标与策略梯度是一致的。

接下来我们将结合部分的代码深入理解PPO算法如何在$RLHF$中大展身手，如何完成大语言模型对齐的任务。

3.2.1 DeepSpeed-Chat 源码解析

3.2.1.1 SFT

3.2.1.2 Reward

3.3.1.3 ppo

3.3.2

3.3 DPO

强化学习的优化目标可以用如下一般形式的公式进行概述：

$$\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}\left[r_\phi(x,y)\right]-\beta\mathbb{D}_{\mathrm{KL}}\left[\pi_\theta(y\mid x)\|\pi_{\mathrm{ref}}(y\mid x)\right], \tag{3.3.1}$$

即让新策略在不偏离原始策略太多的情况下尽可能地生成奖励模型认为较好的内容，DPO将此优化目标进行变体：

$$\begin{aligned}
&\arg\max_\theta \mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}\left[r_\phi(x,y)\right]-\beta\mathbb{D}_{\mathrm{KL}}\left[\pi_\theta(y\mid x)\|\pi_{\mathrm{ref}}(y\mid x)\right]\\
&\quad=\arg\max_\theta \mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}\left[r_\phi(x,y)\right]-\beta\pi_\theta(y\mid x)\log\frac{\pi_\theta(y\mid x)}{\pi_{\mathrm{ref}}(y\mid x)}\\
&\quad\propto\arg\max_\theta \mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}\left[\left[\frac{1}{\beta}r_\phi(x,y)\right]-\log\frac{\pi_\theta(y\mid x)}{\pi_{\mathrm{ref}}(y\mid x)}\right]\\
&\quad=\arg\min_\theta \mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}\left[\log\frac{\pi_\theta(y\mid x)}{\pi_{\mathrm{ref}}(y\mid x)}-\frac{1}{\beta}r_\phi(x,y)\right]
\end{aligned} \tag{3.3.2}$$

式子$\frac{1}{\beta}r_\phi(x,y)$和$\theta$无关，我们可以将其适当地改造变形，即：

$$\frac{1}{\beta}r_\phi(x,y)=\log e^{r_\phi(x,y)/\beta} \tag{3.3.3}$$

故优化目标变为：

$$\begin{aligned}
&\arg\min_\theta \mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}\left[\log\frac{\pi_\theta(y\mid x)}{\pi_{\mathrm{ref}}(y\mid x)}-\frac{1}{\beta}r_\phi(x,y)\right]\\
&=\arg\min_\theta \mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}\left[\log\frac{\pi_\theta(y\mid x)}{\pi_{\mathrm{ref}}(y\mid x)e^{r_\phi(x,y)/\beta}}\right]
\end{aligned} \tag{3.3.4}$$

此时，优化目标中log的分母已经不再是一个概率分布了，我们可以对分母进行归一化：

$$\begin{aligned}
&Z(x)=\sum_y \pi_{\mathrm{ref}}(y\mid x)e^{r_\phi(x,y)/\beta},\\
&\arg\min_\theta \mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}\left[\log\frac{\pi_\theta(y\mid x)}{\pi_{\mathrm{ref}}(y\mid x)e^{r_\phi(x,y)/\beta}}\right]\\
&\quad=\arg\min_\theta \mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}\left[\log\frac{\pi_\theta(y\mid x)}{\left(\pi_{\mathrm{ref}}(y\mid x)e^{r_\phi(x,y)/\beta}/Z(x)\right)Z(x)}\right]\\
&\quad=\arg\min_\theta \mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}\left[\log\frac{\pi_\theta(y\mid x)}{\pi^*(y\mid x)}-\log Z(x)\right]\\
&\quad\Leftrightarrow\arg\min_\theta \mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}\left[\log\frac{\pi_\theta(y\mid x)}{\pi^*(y\mid x)}\right]\\
&\quad=\arg\min_\theta \mathbb{E}_{x\sim\mathcal{D}}\left[\mathbb{D}_{\mathrm{KL}}(\pi_\theta(y\mid x)||\pi^*(y\mid x))\right]
\end{aligned} \tag{3.3.5}$$

当$\pi_\theta(y\mid x)=\pi^*(y\mid x)$时有最小值，即最优概率分布为$\pi^*$。而$\pi^*(y\mid x)$和$r(x,y)$的关系为：

$$r(x,y)=\beta\log\frac{\pi^*(y\mid x)}{\pi_{\text{ref}}(y\mid x)}+\beta\log Z(x) \tag{3.3.6}$$

我们希望对于两个分得出好坏的回答$y_1\succ y_2$，奖励模型的输出有$r(x,y_1)>r(x,y_2)$，Bradley-Terry模型只考虑两个回答得分的差值，即希望$r(x,y_1)>r(x,y_2)$，且好的回答大于坏的回答的**几率**有：

$$\begin{aligned}p^*(y_1\succ y_2\mid x)&=\frac{1}{1+\exp\left(r(x,y_1)-r(x,y_2)\right)}\\&=\frac{1}{1+\exp\left(\beta\log\frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)}-\beta\log\frac{\pi^*(y_2|x)}{\pi_{\text{ref}}(y_2|x)}\right)}\end{aligned} \tag{3.3.7}$$

我们可以通过极大似然的方式来优化策略π_θ，即：

$$\begin{aligned}\mathcal{L}(\pi_\theta;\pi_{\text{ref}})&=-\log\prod_{x,y_w,y_l\sim\mathcal{D}}p^*(y_w\succ y_l\mid x)\\&=-\mathbb{E}_{x,y_w,y_l\sim\mathcal{D}}\left[\log\sigma(\beta\log\frac{\pi_\theta(y_w\mid x)}{\pi_{\text{ref}}(y_w\mid x)}-\beta\log\frac{\pi_\theta(y_l\mid x)}{\pi_{\text{ref}}(y_l\mid x)})\right]\end{aligned} \tag{3.3.8}$$

此时，我们只需要两个模型了，一个$actor$和一个$reference$。原来的强化学习过程就转化成了SFT的形式。

3.3.1 DeepSpeedChat源码解析

这里笔者着重讲解框架中的损失函数计算部分，分为如下三部分

3.3.1.1 构建label mask

```
460            batch_size = batch['input_ids'].shape[0] // 2 #batch:[2*B,S]
461            chosen_input_ids = batch['input_ids'][:batch_size] #[B,S]
462            rejected_input_ids = batch['input_ids'][batch_size:] #[B,S]
463            label_mask = (batch['input_ids'] != tokenizer.pad_token_id).int() #[2B,S]
464            print(f"chosen input ids:{chosen_input_ids}")
465            for i in range(batch_size):
466                divergence_ind = (chosen_input_ids[i] !=
467                                  rejected_input_ids[i]).nonzero().squeeze(-1)
468                if len(divergence_ind) > 0:
469                    divergence_ind = divergence_ind[0]
470                else:
471                    divergence_ind = 0
472                label_mask[i][:divergence_ind] = 0
473                label_mask[i + batch_size][:divergence_ind] = 0
```

如果开发者将batchsize设置成B，由于每一个$prompt$对应了一好一坏的回答，则一共会有$2B$个样本在一次迭代$step$中，前B个样本是好的回答，后B个样本是被拒绝的回答。而$label_mask$则用于选择对应$token$位置的奖励，即$label_mask=1$的位置会被考虑，其他为0的位置不会被纳入计算，由于奖励只是针对模型生成内容的部分，且对于模型的$prefilling$而言，前k个$token$一致时每个位置输出的logits都一致，所以要找到$chosen$与$reject$的第一个顺序遇到不同的$token$的位置索引，将$label_mask$该索引前元素都置为0，实现对应于代码第465-473行。第463行先是把不为$<pad>$的部分置1，再接着把$prompt$和$answer$中前k个相同$token$对应位置的$label_mask$置0。

3.3.1.2 计算对数概率

详见代码第$474-485$行与函数def_batch_logps。

```
474         outputs = model(**batch, use_cache=False)
475         with torch.no_grad():
476             ref_outputs = ref_model(**batch)
477
478         logps = get_batch_logps(outputs.logits, batch['input_ids'],
479                                 label_mask)
480         ref_logps = get_batch_logps(ref_outputs.logits, batch['input_ids'],
481                                     label_mask)
482         chosen_logps = logps[:batch_size]
483         rejected_logps = logps[batch_size:]
484         ref_chosen_logps = ref_logps[:batch_size]
485         ref_rejected_logps = ref_logps[batch_size:]
```

```
218 # Reference: https://github.com/huggingface/trl/blob/main/trl/trainer/dpo_
219 def get_batch_logps(logits, input_ids, label_mask):
220     labels = input_ids.clone() * label_mask #只有对应的answer部分
221     assert logits.shape[:-1] == labels.shape, \
222         "Logits (batch and sequence length dim) and labels must have the s
223     labels = labels[:, 1:]
224     label_mask = label_mask[:, 1:]
225     logits = logits[:, :-1, :] #[B,S-1,H]
226     per_token_logps = torch.gather(logits.log_softmax(-1),
227                                    dim=2,
228                                    index=labels.unsqueeze(2)).squeeze(2)
229     return (per_token_logps * label_mask).sum(-1)
230
231
```

我们需要先将$label$与$label_mask$对应位置元素相乘，只剩下需要考虑的位置的$token$，因此将logits序列对应$token$位置的logit $\in \mathbf{R}^{|V|\times 1}$向量中该$token$的索引的分量，即$\pi_\theta(y_j \mid x), j=1,\ldots,|y|$取出，通过torch. gather函数实现，最终再相加作为整句话的奖励，笔者该处的符号表述多了一个下角标j，和DPO论文中的符号不太一致，原因是Deepspeed-chat框架中该处的奖励模型不是最朴素的奖励模型，而是Outcome Reward模型，即考虑$token$位置的奖励而非只考虑整句话最后一个位置的奖励。

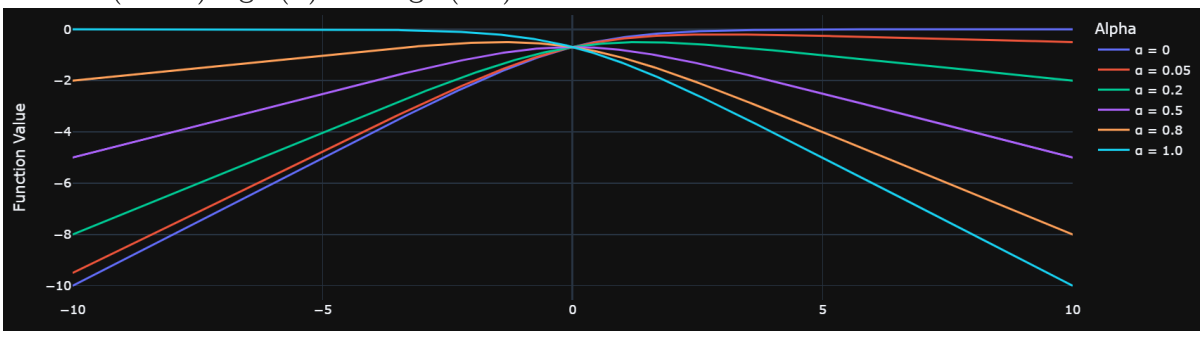
3.3.1.3 计算损失函数

```
487         logits = args.beta * ((chosen_logps - ref_chosen_logps) -
488                               (rejected_logps - ref_rejected_logps))
489         loss = (- torch.nn.functional.logsigmoid(logits) * (1 - args.label_smoothing) - \
490                 torch.nn.functional.logsigmoid(-logits) * args.label_smoothing).mean(0)
491         if args.print_loss:
492             print(
493                 f"Epoch: {epoch}, Step: {step}, Rank: {torch.distributed.get_rank()}, loss = {loss}"
494             )
495         model.backward(loss)
496         model.step()
```

$actor$与$reference$的对数概率变量logps和ref_logps都计算完成后就可以根据公式$3.3.8$带入进行损失函数计算，σ函数内的变量对应图中代码的第$487-489$行。此外，框架中还对损失函数进行了平滑操作：

$$\log\sigma(x) \mapsto (1-\alpha)\log\sigma(x) + \alpha\log\sigma(-x) \tag{3.3.9}$$

函数$(1-\alpha)\log\sigma(x)+\alpha\log\sigma(-x)$的图像如下：



当$\alpha=0$时，为$\log\sigma(x)\in(-\infty,0)$，而随着$a$的增大，函数右边有往下的趋势，函数左边有往上抬的趋势，当$\alpha=1$时，函数和$\log\sigma(x)$图像完全反了过来（关于y轴对称）。即在训练中，虽然我们希望$\log\dfrac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\dfrac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}$大于0，但实际情况可能是小于0的，越远离0则损失函数越大，可能导致训练不稳定，因此可以通过平滑系数对原函数进行趋势的控制。

如上便是Deepspeed-Chat框架的DPO实现代码，关键部分相较于PPO便于理解，实现起来也较为简单，同时训练成本也更低，当然

3.4 GRPO

参考文献

[X]Policy Gradients: The Foundation of RLHF

[X]Proximal Policy Optimization (PPO): The Key to LLM Alignment

[X](WIP) A Little Bit of Reinforcement Learning from Human Feedback

[x]Policy Gradient Algorithms,Weng, Lilian,liliangweng.github.io,2018

[x]深度强化学习（三）：TRPO（Trust Region Policy Optimization，信赖域策略优化）,Dreammaker